

Terminal

Architecture, Data Sources & Open-Source Security Guide

Created by **Oscar Zarraga Perez**

A technical reference describing how the project collects, caches, and serves market data — written so the source code can be released publicly without exposing credentials or personal information.

Copyright © 2026 Oscar Zarraga Perez · All rights reserved

Document Version: 1.2

Released without warranty. See next page.

Disclaimer & Limitation of Liability

No warranty. The Terminal project is provided “AS IS” and “AS AVAILABLE”, without warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose, accuracy, completeness, non-infringement, or continued availability of any upstream data source. The project is a personal-use, educational software experiment. Nothing in this document or in the source code creates a contract, an engagement, or a service-level commitment of any kind.

Not investment advice. The data displayed by the project is aggregated from third-party public sources whose accuracy, timeliness, and continuity cannot be guaranteed. The project is not a broker-dealer, investment adviser, financial planner, accountant, auditor, or tax adviser. None of the output of the project, of this document, or of the source code constitutes investment, legal, accounting, tax, or financial advice. Any decision to buy, sell, hold, or evaluate a security is the sole responsibility of the person making that decision and should be taken only after consulting an appropriately licensed professional.

Limitation of liability. To the maximum extent permitted by applicable law, the authors and contributors of the project disclaim all liability for any direct, indirect, incidental, special, consequential, exemplary, or punitive damages — including but not limited to lost profits, lost data, trading losses, missed opportunities, regulatory fines, reputational harm, or any other commercial damage — arising out of or in connection with the use, misuse, or inability to use the project, the documentation, the source code, or any data obtained through the project, even if advised of the possibility of such damages.

Operator responsibility. Anyone who downloads, runs, modifies, redistributes, or deploys the project (the “operator”) is solely responsible for: (a) compliance with the terms of service of every upstream data source the project queries; (b) compliance with all applicable laws, including securities, market-data, privacy, export, and tax laws in the operator’s jurisdiction; (c) the security of any API keys, credentials, or other secrets the operator chooses to configure; (d) any consequences arising from public, commercial, or multi-user deployments of the project.

Third-party services. The project queries third-party endpoints. It does not control, endorse, or warrant any third-party content, service availability, accuracy, or terms. Endpoints may change, rate-limit, or cease without notice. Trademarks, service marks, and logos referenced in this document are the property of their respective owners; references are nominative and do not imply endorsement, affiliation, or sponsorship in either direction.

By using the project in any form, the operator acknowledges and agrees to all of the above. Operators who do not agree must not use, fork, or redistribute the project.

Authorship. Terminal was created and authored by **Oscar Zarraga Perez**. Copyright © 2026 Oscar Zarraga Perez. Released under the MIT License. Attribution to the author must be preserved in all copies and substantial portions of this software, including in source-file headers, in the rendered application footer, and in this disclaimer page. Removing or obscuring the attribution violates the MIT License under which this project is distributed.

1. Executive Summary

Terminal is a single-file Python backend serving a single-file HTML/JS frontend. The backend (`server.py`) speaks plain `http.server` and aggregates data from roughly twenty public sources — four optional keyed APIs and a longer list of free, documented endpoints. Every keyed source is treated as a redundancy layer: the project runs end-to-end with no keys configured at all.

The design priority is resilience. Each request fans out across multiple sources in parallel, falls back through a documented chain on rate-limits, and caches results in-memory with conservative TTLs. A 60-second per-host cooldown is triggered automatically on any HTTP 429/999 so that a single noisy upstream cannot break a quote.

This document describes every data source, the fallback ordering, the caching model, and — most importantly — the steps required to publish the source code openly without leaking credentials or personal data.

Read first: Section 11 (Open-Source Security Checklist). Every item on that list should be confirmed before the repository is pushed to GitHub or any other public host.

2. Architecture Overview

2.1 Process Layout

The project is two files in production:

- `server.py` — the backend, ~5,500 lines of pure Python. Spawns a `ThreadingHTTPServer` bound to `127.0.0.1:8788` by default.
- `terminal.html` — the frontend, single-file HTML/CSS/JS served at `/`. Calls back into the Python server through a small JSON API that mirrors Yahoo Finance's schema where convenient.
- `keys.json` — optional, gitignored. Holds the operator's free-tier API keys. Loaded at startup and hot-reloaded every two seconds when the file's modification time changes.

2.2 Runtime Stack

Standard library only — no third-party Python packages required to run.

HTTP server	<code>http.server.ThreadingHTTPServer</code>
Concurrency	<code>threading + concurrent.futures.ThreadPoolExecutor</code> for parallel fan-out
URL/JSON	<code>urllib.request, urllib.parse, json</code>
XML/RSS	<code>xml.etree.ElementTree, email.utils.parsedate_to_datetime</code>
Parsing	<code>re, html</code> for tag stripping
CLI flags	<code>argparse</code> — <code>--port</code> , <code>--host</code> , <code>--no-browser</code> , <code>--tab</code>

2.3 Request Lifecycle

A typical browser tick goes through five stages:

- **Route:** `Handler.do_GET()` dispatches by URL path to a route handler (e.g. `/api/quote`, `/api/chart/SYM`).
- **Cache check:** `_cache_get(key)` returns a memoized response if it is still within TTL.
- **Fan-out:** `_parallel(tasks, timeout=12)` schedules every source on a thread pool with a per-task timeout.
- **Fallback merge:** results are merged in priority order; any source that timed out, returned empty, or hit a cooldown is skipped silently.
- **Cache write:** the merged response is written back to `_cache` with a TTL appropriate to its kind (see Section 9).

3. Keyed API Sources (All Optional)

Four free-tier providers can be plugged in for redundancy and depth. Every one of these is optional. The project runs with zero keys. Each configured key unlocks an additional fallback layer in the rotation chain.

All keys are loaded from `keys.json` (local, gitignored) or from environment variables. Environment variables take precedence over the file on most platforms, which is the safer choice for shared or public deployments.

Provider	Coverage	Free-tier quota	Env var(s)
Alpha Vantage	Quotes, intraday & daily OHLC, company overview, fundamentals	5 req/min · 500/day	ALPHAVANTAGE_API_KEY · ALPHA_VANTAGE_KEY
Financial Modeling Prep	Quotes, history, intraday, profile, ratios, news	250/day	FMP_API_KEY
Finnhub	Quotes, candles, news, analyst recs, earnings, profile	60 req/min	FINNHUB_API_KEY
Twelve Data	Quotes, time-series, chart, statistics — all asset classes	8 req/min · 800/day	TWELVE_DATA_API_KEY · TWELVEDATA_API_KEY

Each keyed provider is gated behind a dedicated endpoint to preserve its daily quota; the heavy traffic of `/api/quote` and `/api/chart` is served by free sources first and only falls through to a keyed provider if everything earlier in the chain has been exhausted or cooled down.

3.1 Where to register

- Alpha Vantage — alphavantage.co/support/#api-key
- Financial Modeling Prep — site.financialmodelingprep.com/developer
- Finnhub — finnhub.io/register
- Twelve Data — twelvedata.com/pricing

3.2 Key file format

The repository ships `keys.sample.json` with empty values. The file is copied to `keys.json` (which is in `.gitignore`) and the operator pastes their own keys.

```
{
  "_comment": "All fields are optional. The project runs with zero keys.",
  "alpha_vantage": "",
  "fmp": "",
  "finnhub": "",
  "twelve_data": ""
}
```

4. Unkeyed Sources

These are the public, documented endpoints the project hits without any credentials. Each is wrapped by a polite-fetch helper that enforces a per-host minimum interval and a 60-second cooldown on rate-limit errors. Use of each endpoint is subject to the upstream service's terms; see Section 12.

4.1 Market Data

Host	Returns	Wrapper
query1/2.finance.yahoo.com	v7 batch quotes, v8 chart, quoteSummary modules	yahoo_v7_quotes_batch · yahoo_v8_quote · yahoo_chart_api · yahoo_chart_raw
stooq.com	CSV quotes, daily/weekly history, 52-week stats	stooq_quote · stooq_history · stooq_year_stats
api.coingecko.com	Crypto price, OHLC	cg_price · cg_ohlc
api.nasdaq.com	Earnings calendar, price targets, ratings, dividend history, institutional holdings	nasdaq_next_earnings · nasdaq_price_targets · nasdaq_ratings · nasdaq_earnings_surprise · nasdaq_earnings_forecast · nasdaq_institutional_holdings · nasdaq_dividend_history
stockanalysis.com	Chart history JSON, fundamentals & forecasts (HTML scrape)	stockanalysis_history · sa_statistics · sa_forecast · sa_ratings · sa_institutional · sa_insider · sa_financials
finviz.com	Quote & earnings (HTML fallback)	finviz_quote · _finviz_quote_fallback · _finviz_earnings_date
cdn.cboe.com	Historical options volatility data	cboe_history

4.2 SEC / Regulatory

Host	Returns	Wrapper
www.sec.gov	company_tickers.json (ticker→CIK), filing archives	sec_get_cik · _sec_fetch · sec_submissions · _press_sec_8k
data.sec.gov	XBRL company facts (income statement, balance sheet, cashflow), submissions JSON	sec_company_facts · sec_submissions
efts.sec.gov	Full-text search against EDGAR	(used opportunistically through sec_get_cik)

4.3 News, Press & Sentiment

Host	Returns	Wrapper
news.google.com	Google News RSS by query	_press_google_news · fetch_news

Host	Returns	Wrapper
feeds.finance.yahoo.com	Yahoo Finance RSS per ticker	yahoo_rss_news
www.globenewswire.com	GlobeNewswire press-release RSS by company slug	_press_globenewswire_rss
api.stocktwits.com	Streams + bullish/bearish sentiment ratio	stocktwits_stream
finance.yahoo.com	Press-releases tab scrape (<code>__NEXT_DATA__</code> JSON)	_press_yahoo
finance.yahoo.co.jp	Tokyo Stock Exchange disclosures (TDnet) for Japanese tickers / their ADRs	_press_yahoo_japan_disclosures
kabutan.jp	Per-stock disclosure tables (TDnet aggregation)	_press_kabutan

5. Quote & Chart Fallback Chain

Both the quote and chart pipelines walk an ordered list of sources. The first source that returns a complete record wins. Sources currently in cooldown (60-second penalty after a 429/999) are skipped silently and the chain advances to the next.

5.1 Quote Order (shape_quote)

```
Crypto detected? → CoinGecko
Yahoo v7 batch → Yahoo v8 chart-meta + Stooq blend → Stooq CSV
→ Twelve Data (if key) → FMP (if key) → Finnhub (if key) → Alpha Vantage (if key)
→ Yahoo HTML scrape → Finviz scrape → StockAnalysis scrape (intl/OTC)
```

5.2 Chart Order (shape_chart)

```
Crypto? → CoinGecko OHLC
Yahoo v8 chart → Stooq history (daily / weekly / monthly only)
→ Nasdaq historical → StockAnalysis history JSON
→ Cboe history (options-vol historical)
```

5.3 The Polite-Fetch Layer

Every outbound HTTP call goes through `_polite_fetch(url, ...)`. Three jobs:

- **Throttle.** Per-host minimum interval enforced via `HOST_MIN_INTERVAL` (e.g. Yahoo 0.35s, Stooq 0.2s, Finviz 0.5s).
- **UA rotation.** A small pool of common desktop user-agents is rotated to avoid sticky bot fingerprints.
- **Cooldown.** Any 429 or 999 response triggers `_host_cooldown(host, 60)`. Subsequent calls to that host short-circuit until the cooldown expires.

Errors are *caught and logged* to `stderr`; they never propagate to the HTTP response. A failed source becomes an empty result and the next source in the chain fires.

6. Press Release Pipeline

The press tab is the most aggressive fan-out in the system. Six sources fire in parallel; results are merged in priority order, deduped by URL and title, and filtered to ensure each item belongs to the right company.

6.1 Sources (in merge priority)

Source	Function	Coverage
TDnet · Yahoo Japan	<code>_press_yahoo_japan_disclosures</code>	Tokyo-listed companies + ADRs
TDnet · Kabutan	<code>_press_kabutan</code>	Tokyo-listed companies
Hardcoded IR page	<code>_press_ir_website</code>	Per-ticker IR-page mapping (PRESS_IR_PAGES)
Yahoo Finance	<code>_press_yahoo</code>	US-listed press-release stream
GlobeNewswire RSS	<code>_press_globenewswire_rss</code>	Company-name slug RSS
Google News RSS	<code>_press_google_news</code>	Filtered by ticker + company name

6.2 Foreign-Ticker Routing

Tickers that don't exist on US exchanges (e.g. MTPLF, the OTC ADR for the Tokyo-listed 3350 . T) are mapped through FOREIGN_TICKER_MAP. The map carries the home-exchange ticker, a StockAnalysis slug (e.g. tyo/3350), country, currency, and a static Yahoo-shaped `assetProfile` so the profile tab renders correctly even when SEC and Yahoo have no record.

When the press fan-out runs for a foreign ticker, `_press_yahoo_japan_disclosures` and `_press_kabutan` resolve the home ticker and fetch the official TDnet disclosure feed for that company — the same feed used by Japanese brokerages.

6.3 Keep Filter

Items keep if *any* of:

- Source is a hardcoded IR page (URL is authoritative).
- Source is a TDnet feed (regulatory authority).
- Title contains the ticker as a word boundary, the full company name, or a distinctive cleaned form of the name (suffixes stripped).

Short generic names (“Apple”, “Tesla”) are required to match as a whole word to avoid false positives like “Apple Hospitality”. Items are deduped by URL (without query string) and lowercased title.

7. SEC Filings Pipeline

7.1 CIK Lookup

`sec_get_cik(ticker)` fetches `www.sec.gov/files/company_tickers.json` exactly once per process and caches the resulting ticker $\rightarrow$ (10-digit-CIK, company-name) map behind a thread lock. Foreign-listed tickers that aren't in this map fall through to `FOREIGN_TICKER_MAP`.

7.2 Submissions / Recent Filings

`sec_submissions(cik)` calls `data.sec.gov/submissions/CIK{cik}.json` (cached six hours). The recent-filings array is parsed for: form type (8-K, 8-K/A, 6-K, 10-K, 10-Q), accession number, filing date, primary document filename, and 8-K item codes.

7.3 Document URLs

```
https://www.sec.gov/Archives/edgar/data/{cik}/{accession_no_dashes}/{primary_document}
```

If the primary-document field is missing the link falls back to the browse-Edgar index page for that accession. Item codes are surfaced in the press-release title (e.g. "8-K Filing · Item(s): 2.02").

7.4 XBRL Company Facts

For income statement, balance sheet, and cash flow, the pipeline calls `data.sec.gov/api/xbrl/companyfacts/CIK{cik}.json` (cached six hours). `_extract_concept()` walks the XBRL tags (`us-gaap:Revenues`, `us-gaap:NetIncomeLoss`, etc.) and pivots them into per-period values for the financials tabs.

7.5 SEC Compliance

The SEC requires a descriptive **User-Agent** string identifying the operator. `_sec_fetch()` sends a compliant UA. **Forks of this project must change that UA** to identify the deploying party so the SEC can reach a contact if traffic looks anomalous. The default UA is a generic placeholder and must be replaced; the project provides the mechanism but cannot warrant the operator's compliance.

8. News & Sentiment

Three different news endpoints sit on the HTTP surface, each tuned for a different use case:

Endpoint	Behaviour
<code>/api/news/{sym}</code>	Single-source Google News RSS, fastest, lightest. Default for the ticker NEWS tab on quick-scan loads.
<code>/api/news-all/{sym}</code>	Parallel fan-out: Yahoo Finance RSS + GlobeNewswire RSS + Google News + (Finnhub + FMP if their keys are loaded). Deduped by URL and title.
<code>/api/press/{sym}</code>	Press-release-specific aggregator described in Section 6.

8.1 Sentiment

`/api/sentiment/{sym}` wraps the public StockTwits messages API and returns a normalized stream plus a bullish-vs-bearish ratio computed over the last n tagged messages. The ratio is a raw aggregation of public posts and is not a signal, recommendation, or model output.

9. Caching & Throttling

9.1 In-Memory Cache

All caching is in-process. The `_cache` dict is keyed by string and holds `(expire_ts, value)` tuples; a `threading.Lock` guards reads and writes. There is no disk persistence, no Redis, nothing to delete when shutting down. Restarting the server gives a clean cache.

9.2 TTLs by Category

Category	TTL
Live quote	15–30 seconds
Chart history	30–120 seconds
News (any)	2–5 minutes
Press releases	5 minutes
HTML scrapes	10 minutes – 2 hours
Keyed-provider endpoints	1 – 24 hours
SEC submissions / facts	6 hours
Insider transactions	2 hours
Dividend history	24 hours
Stooq full-year stats	24 hours

9.3 Per-Host Throttling

`HOST_MIN_INTERVAL` defines the minimum spacing between requests to each host. Conservative defaults shipped:

Host pattern	Min interval
query?.finance.yahoo.com	0.35s
finance.yahoo.com (HTML)	0.40s
stockanalysis.com	0.40s
stooq.com / api.coingecko	0.20s
api.binance.com	0.20s
data.sec.gov / sec.gov	0.20s
finviz.com	0.50s
api.stocktwits.com	0.50s
news.google.com / RSS feeds	0.30s

`_host_cooldown(host, 60)` overlays a 60-second blanket cooldown on top whenever a host returns 429 or 999.

10. HTTP API Surface

Every endpoint returns `application/json; charset=utf-8` with `Cache-Control: no-store` and `Access-Control-Allow-Origin: *`. The wildcard CORS is acceptable here only because the server binds to localhost by default. Operators who change the bind address must tighten CORS — see Section 11.4.

Path	Returns
GET /	Serves terminal.html
GET /api/quote?symbols=A,B,C	Multi-symbol Yahoo-shaped quotes
GET /api/chart/{sym}?range=&interval;=	Chart OHLC time-series
GET /api/summary/{sym}	quoteSummary modules (profile, fundamentals, key stats, etc.)
GET /api/financials/{sym}	Income / balance / cashflow / earnings — SEC + keyed providers
GET /api/search?q=	Ticker + company-name search
GET /api/news/{sym}	Google News RSS only (fast)
GET /api/news-all/{sym}	Multi-source aggregated news
GET /api/press/{sym}	Aggregated press releases
GET /api/sentiment/{sym}	StockTwits sentiment
GET /api/peers/{sym}	SEC peer companies
GET /api/insider/{sym}	Insider transactions
GET /api/dividends/{sym}	Dividend history
GET /api/yearstats/{sym}	Stooq full-year stats
GET /api/stats/{sym}	Alpha Vantage overview
GET /api/profile-ext/{sym}	FMP extended profile + ratios
GET /api/ratings/{sym}	Finnhub recommendations + earnings
GET /api/statistics/{sym}	Twelve Data statistics
GET /api/keys/status	Which keyed providers are loaded (no key values returned)
GET /api/keys/reload	Hot-reload keys.json
GET /api/diag?sym=	Reachability + cooldown diagnostics

11. Open-Source Security Checklist

Every item on this list should be confirmed *before* the first push to a public Git host. Most credential leaks happen in the very first commit and remain searchable on GitHub forever, even after deletion. The project supplies the mechanism; the operator is responsible for completing the checklist.

11.1 Pre-Publish Checklist

- **Delete** `keys.json` from the working tree, or move it outside the repo entirely. Confirm it is listed in `.gitignore`.
- **Ship only** `keys.sample.json` with every value set to an empty string.
- Run `git log -p` and search for any string matching `[A-Za-z0-9]{20,}`. If a real key was ever committed, treat it as compromised — rotate it immediately at the provider before publishing.
- Use `git filter-repo` (or the BFG Repo-Cleaner) to scrub historical commits if needed. *Force-pushing alone does not remove leaked secrets from GitHub's caches.*
- Search the codebase for any personal name, email address, employer, or other PII. None should appear in source, comments, or sample data.
- Search for absolute filesystem paths (`/Users/...`, `C:\\...`, `/home/...`). Replace with relative paths or remove.
- Search for hardcoded IP addresses or internal hostnames. Replace with `127.0.0.1` / `localhost`.
- Verify the `SEC User-Agent` string is generic (`"ProjectName admin@example.com"`) and does not carry any personal email.
- Strip any test or debug print statements that echo user input or environment variables.

11.2 .gitignore (minimum)

```
keys.json
*.local.json
.env
.env.*
__pycache__/
*.pyc
.DS_Store
Thumbs.db
```

11.3 Recommended Pre-Commit Hook

Installing one of the scanners below as a pre-commit hook causes a leaked key to be caught before it can reach the remote:

- **gitleaks** — single-binary scanner, fast, zero-config defaults. github.com/gitleaks/gitleaks
- **trufflehog** — entropy-based detector with active validation against provider APIs. github.com/trufflesecurity/trufflehog
- **detect-secrets** — Python-native, Yelp-maintained. github.com/Yelp/detect-secrets

11.4 Runtime Hardening

- Keep the bind address at `127.0.0.1`. The `CORS *` header is only safe because nothing routable on the network can reach the server.
- If the server is ever exposed to a LAN or the internet, it must be placed behind a reverse proxy that requires authentication. A CSRF/origin check should be added, traffic should be rate-limited per IP, and the CORS header should be tightened to a known origin.
- Do *not* log the contents of `keys.json` or any environment variable. The current code logs only key *names* (“alpha_vantage loaded”) — that posture should be preserved.
- Treat the `/api/diag` endpoint as diagnostic-only. It reveals which sources are configured and any active cooldowns; it should be gated behind a token or removed in production builds.
- API keys should be rotated at least quarterly, and immediately on any suspicion of exposure.

11.5 If a Key Leaks

- **Rotate first, clean up second.** The exposed key should be revoked at the provider and a new one generated and pasted into the local `keys.json`.
- Only after rotation should the git history be scrubbed (`git filter-repo`) and force-pushed.
- The provider should be notified if the leak might have allowed third-party usage; most have an abuse channel.
- Provider billing should be audited for the exposure window.

12. Licensing, Attribution & Legal Posture

12.1 Upstream Terms of Service

Each upstream service has its own terms. The list below is summary, non-exhaustive, and not legal advice. The operator must read the current terms of every service the project queries before deploying:

- **Yahoo Finance** endpoints are unofficial. Many open-source projects use them with the same throttle/UA pattern this project uses, but Yahoo may block traffic at any time. The README should document this clearly.
- **SEC EDGAR** requires a descriptive User-Agent per its fair-access policy. The header is non-negotiable and must identify the deploying party.
- **StockTwits** messages API is public but rate-limited; bulk caching or rehosting their content may breach their terms.
- **Stooq, StockAnalysis, Finviz, Kabutan, Nasdaq** — each has commercial-use restrictions. The project as shipped is for personal and educational use only; any commercial offering requires a license or a replacement paid feed.
- The four **keyed providers** all expose free tiers explicitly intended for personal and prototype use. Each provider's ToS is the binding document; quotas and permitted uses change over time.

12.2 Recommended Source Licence

Recommended licence for the Terminal source itself: **MIT** or **Apache-2.0**. Both include explicit no-warranty and limitation-of-liability clauses. A `DISCLAIMER.md` with the same content as the disclaimer page in this document should be added at the repo root.

12.3 No-Warranty / No-Advice Statement

The project is distributed without warranty of any kind. Nothing in the project, the documentation, or the source code constitutes investment, legal, accounting, tax, or financial advice. The data shown is aggregated from third-party public sources whose accuracy, timeliness, and continuity cannot be guaranteed. The authors and contributors disclaim all liability for any damages arising from use of the project to the maximum extent permitted by applicable law. Operators are solely responsible for their own use of the project, including compliance with upstream terms of service and applicable law.

12.4 Trademarks

All product names, trademarks, and registered trademarks referenced in this document or in the source code are the property of their respective owners. References are nominative and do not imply endorsement, sponsorship, or affiliation.

Appendix A — Recommended Repo Skeleton

```
terminal/
■■■ server.py
■■■ terminal.html
■■■ keys.sample.json
■■■ README.md
■■■ LICENSE
■■■ DISCLAIMER.md
■■■ .gitignore
■■■ docs/
■   ■■■ ARCHITECTURE.pdf    # this document
■   ■■■ KEYS_SETUP.md
■■■ scripts/
    ■■■ run.sh
    ■■■ run.bat
```

Notable absences: no `keys.json`, no `.env`, no internal IP addresses, no employer or personal information anywhere in the tree. Every secret lives in a file the `gitignore` explicitly excludes or in environment variables that the code reads at runtime.

Appendix B — Build & Run

From a fresh clone, no keys configured:

```
$ python3 server.py
[keys] loaded: (none – running with public sources only)
Listening on http://127.0.0.1:8788
```

With keys, after copying the sample to `keys.json` and pasting in values *or* exporting them as env vars:

```
$ export FMP_API_KEY=your_key_here
$ export FINNHUB_API_KEY=your_key_here
$ python3 server.py --port 8788
[keys] loaded: fmp, finnhub
Listening on http://127.0.0.1:8788
```

End of document. Section 11 is a hard gate: the repository should not be made public until every checkbox is satisfied. The disclaimer on page 2 governs all use of the project.